

Play My Jam

Spotify'dan YouTube Altyapısına Geçiş

Teknik Analiz, Fizibilite ve Detaylı Geçiş Planı

Hazırlanma tarihi: 10 Temmuz 2026 · Versiyon 1.0

Bu doküman mevcut PMJ kod tabanının (Next.js 16 + Supabase + Vercel) incelenmesine ve güncel YouTube API dokümantasyonuna dayanır. Hukuki bölümler bilgilendirme amaçlıdır, hukuki danışmanlık yerine geçmez.

İçindekiler

Bölüm	Başlık
1.	Yönetici Özeti
2.	Mevcut Mimari: Spotify Entegrasyonunun Anatomisi
3.	YouTube Ekosistemi: Seçenekler ve Kısıtlar
4.	Hedef Mimari
5.	Veritabanı Değişiklikleri
6.	Dosya Bazlı Değişiklik Haritası
7.	Katalog Eşleştirme: Spotify → YouTube
8.	API Kota Yönetimi ve Maliyet
9.	Riskler ve Azaltma Stratejileri
10.	Hukuki Durum (ToS + Türkiye Telif)
11.	Aşamalı Geçiş Planı ve Zaman Çizelgesi
12.	Sonuç ve Öneriler

1. Yönetici Özeti

PMJ bugün müzik çalmayı **mekandaki Spotify uygulamasına** devrediyor: admin cihazında Spotify açık duruyor, backend Spotify Connect API üzerinden bu cihazı uzaktan kumanda ediyor ve 5 saniyede bir durum sorgulayarak (polling) şarkı bitişini tahmin ediyor. Bu yapı üç kronik soruna yol açıyor: her mekan için Premium hesap ve OAuth bağlantısı gerekiyor, Spotify geliştirici uygulaması kullanıcı sayısı/quota extension sürecine takılıyor ve "aktif cihaz yok / token koptu / repeat döngüsü" gibi kırılğanlıklar sync koduna sızmış durumda.

YouTube Music'in resmi bir API'si yoktur ve kuyruk kontrolüne izin veren hiçbir resmi yol sunmaz. Gayri resmi kütüphaneler (ytmusicapi) çerez tabanlıdır, Google ToS'unu ihlal eder ve ticari ürün için uygun değildir. Bu nedenle önerilen geçiş YouTube Music'e değil, **YouTube'un resmi iki API'sine** yapılmalıdır: arama/metadeta için **YouTube Data API v3**, oynatma için **IFrame Player API**.

Bu geçişin mimari sonucu köklü ama olumlu: oynatıcı artık harici bir uygulama değil, **admin panelinin kendisi** olur. Admin cihazındaki tarayıcıda gömülü YouTube player sıradaki videoyu çalar; şarkı bittiğinde player'ın *onStateChange(ENDED)* olayı tetiklenir ve kuyruk deterministik şekilde ilerler. OAuth, token yenileme, cihaz keşfi ve 5 saniyelik polling/sync kodunun tamamı silinir. Mekan başına hesap bağlama derdi biter; her mekan sadece bir tarayıcı sekmesi açar.

İki kritik dikkat noktası vardır: (1) YouTube API kurallarına göre **video görünür olmalıdır** — ses-yalnız/arka plan oynatma yasaktır; admin cihazının ekranı (veya bağlı bir TV) video gösterecek şekilde konumlanmalıdır. (2) Data API'nin arama ucu pahalıdır (100 birim/arama, günlük varsayılan kota 10.000 birim); önce yerel katalogda arama + agresif önbellekleme + kota artış başvurusu şarttır. Hukuki cephede ise durum Spotify'dakiyle aynıdır: hangi platform olursa olsun mekan, umuma iletim için MESAM/MSG/MÜ-YAP lisansı almakla yükümlüdür.

Tahmini toplam efor: **3-4 hafta** (tek geliřtirici), pilot mekan dahil. Aylık ek altyapı maliyeti: **0 TL** (YouTube API'leri kota dahilinde ücretsizdir); mekan başına Spotify Premium maliyeti ortadan kalkar.

2. Mevcut Mimari: Spotify Entegrasyonunun Anatomisi

Kod tabanı incelemesine göre PMJ; Next.js 16 (App Router, Vercel), Supabase (Postgres + Auth + Realtime) ve Spotify Web API üçlüsü üzerine kurulu. Üç rol var: müşteri (QR → /venue/[slug]), mekan admini (/admin/[slug]) ve super admin. Kuyruk, oylama, jeton ekonomisi ve istekler tamamen Supabase'de — **bu katman Spotify'dan bağımsızdır ve geçişte olduğu gibi korunur**. Spotify'a dokunan katmanlar şunlardır:

Bileşen	Ne yapıyor?	Spotify bağımlılığı
lib/spotify.ts (428 satır)	Client-credentials token (arama), mekan bazlı OAuth (authorization code + refresh), getVenueAccessToken, startTrack, playNextFromQueue, getSpotifyAuthUrl, getUserPlaylists	Tam bağımlı — çekirdek modül
/api/spotify/auth + /callback	Mekan Spotify hesabını OAuth ile bağlar, token'ları venues tablosuna yazar	Tam bağımlı — silinecek
/api/spotify/search	Client-credentials token ile Spotify /search (market=TR), sonucu spotify_track_id + metadata olarak döner	Tam bağımlı — YouTube aramasıyla değişecek
/api/spotify/sync/[venueId]	5 sn'de bir Spotify player durumunu çeker; şarkı bitti mi, repeat döngüsüne mi girdi, ilerleme ne — tahmine dayalı kuyruk ilerletme	Tam bağımlı — event tabanlı modelde tamamen gereksizleşir
/api/spotify/player/[venueId]	play/pause/next/previous komutlarını Spotify Connect'e iletir; aktif cihaz bulur (getActiveDeviceId)	Tam bağımlı — yerel player komutlarına dönüşür
/api/spotify/now-playing/[venueId]	Anlık çalan şarkıyı Spotify'dan okur	Tam bağımlı — now_playing tablosundan okunur hale gelir
/api/admin/spotify/playlists + import	Mekanın Spotify çalma listelerini içe aktarır	YouTube playlistItems.list ile yeniden yazılır
Admin dashboard (app/admin/.../page.tsx)	spotifyConnected/spotifyNowPlaying durumu, player kontrolleri, polling	UI yeniden düzenlenir: gömülü player eklenir
songs tablosu	spotify_track_id benzersiz kimlik olarak kullanılıyor (RPC'ler dahil: get_song_user_state)	Şema migrasyonu gerekir (Bölüm 5)
venues tablosu	spotify_access_token, spotify_refresh_token, spotify_token_expires_at, spotify_account_id/name	Kolonlar kaldırılır

2.1 Mevcut yapının kırılabilirlik noktaları

Kodun kendisi bu kırılganlıkları itiraf ediyor: sync route'unda repeat döngüsü tespiti için ilerleme karşılaştırması ("aynı şarkı ama ilerleme geri sarmış → şarkı bitti"), startTrack içinde her çalmada repeat'i kapatma çağrısı, "Spotify'da aktif cihaz yok, Spotify uygulamasını açın" hata yolu ve SpotifyAuthRevokedError ile token temizleme mantığı. Bunların tümü, oynatıcının PMJ'nin kontrolü dışında ayrı bir uygulama olmasının semptomlarıdır. Ayrıca geliştirici uygulaması Development Mode'da kaldığı sürece en fazla 25 test kullanıcısı bağlanabilir; her yeni mekan hesabının panele eklenmesi bu limite ve Spotify'ın quota extension onay sürecine tabidir — kullanıcının yaşadığı "5 mekan" tıkanıklığının kökü budur.

3. YouTube Ekosistemi: Seçenekler ve Kısıtlar

3.1 Seçeneklerin karşılaştırması

Seçenek	Arama	Oynatma/Kuyruk	Resmiyet	PMJ için karar
YouTube Music (resmi)	Yok	Yok — hiçbir public API yok	API yayınlanmamış	İmkansız
ytmusicapi (gayri resmi)	Var (çerez ile)	Kısıtlı, kırılabilir	ToS ihlali, hesap kapatma riski, Google arayüz değiştirince kırılır	Reddedildi
YouTube Data API v3	Var (search.list, videos.list)	Yok (yalnız metadata)	Resmi, ücretsiz kota: 10.000 birim/gün	Arama + metadata için SEÇİLDİ
YouTube IFrame Player API	—	Var: loadVideoById, onStateChange(ENDED), play/pause/seek	Resmi, kota tüketmez	Oynatma için SEÇİLDİ

3.2 IFrame Player API'nin PMJ'ye verdikleri

Player, admin panel sayfasına gömülen bir iframe'dir ve JavaScript ile tam kontrol edilir: *loadVideoById(id)* ile sıradaki şarkı yüklenir, *onStateChange* olayı ENDED (0) döndüğünde şarkı bitmiştir — tahmin yok, polling yok. *getCurrentTime()/getDuration()* ilerleme çubuğunu besler, *onError* (video kaldırılmış, embed kapalı, bölge engelli) durumlarında otomatik olarak sıradaki şarkıya geçilir. Spotify'daki "aktif cihaz" kavramı tamamen ortadan kalkar: cihaz, panelin açık olduğu tarayıcıdır.

3.3 Uyulması zorunlu YouTube API kuralları

Kural	PMJ'ye etkisi
Arka plan / ses-yalnız oynatma yasak (Required Minimum Functionality)	Player sekmesi görünür kalmalı; video gizlenemez, üstü kapatılamaz. Admin cihazı ekranı (veya HDMI ile TV) video gösterir.
Player en az 200×200 px; otomatik oynatma görünür olmadan başlayamaz	Panel tasarımında player'a sabit görünür alan ayrılır (öneri: 480×270 ve üzeri).
Player üzerine overlay/tıklama tuzağı yasak; standart kontroller (ses, altyazı) kısıtlanamaz	Kuyruk/oylama widget'ları player'ın yanına yerleşir, üstüne değil.
Embedded player HTTP Referer göndermeli; API anahtarı proje bazlı	Next.js tarafında zaten sağlanır; API anahtarı yalnız sunucu tarafında tutulur.
Videoların indirilmesi, ayrıştırılması, ayrı ses akışı çekilmesi yasak	yt-dlp benzeri hiçbir yaklaşım kullanılamaz — oynatma yalnızca resmi player ile.
Tarayıcı autoplay politikası (API kuralı değil, pratik kısıt)	Vardiya başında admin bir kez 'Başlat'a tıklar; sonrası kesintisiz zincirlenir.

4. Hedef Mimari

Kuyruk, oylama, jeton ve istek katmanı Supabase'de aynen kalır. Değişen tek şey "çalma motoru"dur: Spotify Connect uzaktan kumandası yerine admin panelde yaşayan bir Player bileşeni gelir.

```
MÜŞTERİ (telefon, QR) ADMIN CİHAZI (mekan)
└─ /venue/[slug] └─ /admin/[slug] → <YouTubePlayer>
| arama isteği ▲ Supabase Realtime (kuyruk değişti)
▼ | onStateChange(ENDED) → sıradaki
/api/search → 1) songs tablosunda ara |
| 2) yoksa YouTube Data API |
▼ ▼
SUPABASE: songs · queue · votes · tokens · now_playing ← ilerleme/durum yazımı
```

4.1 Akışlar

Şarkı isteme: Müşteri arama yapar → önce songs tablosunda tam metin araması (ücretsiz, hızlı) → sonuç yoksa sunucu tarafında YouTube search.list (videoCategoryId=10 Music, regionCode=TR, videoEmbeddable=true) → seçilen video songs tablosuna youtube_video_id ile yazılır → mevcut istek/onay/jeton akışı değişmeden çalışır.

Oynatma döngüsü: Player bileşeni mount olduğunda now_playing + kuyruğu okur → loadVideoById → ENDED olayında /api/player/next çağrılır → backend mevcut playNextFromQueue mantığıyla (Spotify çağrıları çıkarılmış hali) sıradaki şarkıyı seçer, now_playing'i günceller, video kimliğini döner → player yeni videoyu yükler. Müşteri tarafındaki "şimdi çalıyor" ekranı Spotify'a sormak yerine now_playing tablosunu Supabase Realtime ile dinler — mevcut /api/spotify/now-playing polling'i de kalkar.

Kesinti toleransı: Player sekmesi kapanır/çökerse: sayfa yeniden açıldığında now_playing'den kaldığı yeri okur. Bir 'heartbeat' kolonu (now_playing.last_heartbeat_at) admin panelde 'oynatıcı çevrimdışı' uyarısı göstermeyi sağlar. Videosu oynatılamayan (onError 101/150: embed kapalı) şarkılar songs.embeddable=false işaretlenir ve otomatik atlanır.

4.2 Bu mimarinin kazandırdıkları

OAuth ve token yaşam döngüsü tamamen silinir (lib/spotify.ts'in ~%60'ı). Sync polling (5 sn) yerine olay tabanlı, deterministik ilerleme gelir — Vercel function çağrı sayısı ciddi düşer. Mekan kurulumu "Spotify hesabı bağla + Premium al + uygulamayı açık tut" yerine "tarayıcıda paneli aç"a iner. Video gösterimi zorunluluğu, bar/kafe bağlamında klip ekranı olarak ürün değerine dönüştürülebilir.

5. Veritabanı Değişiklikleri

Tek migrasyon dosyasıyla (0008_youtube.sql) yapılabilir; geri alınabilir olması için Spotify kolonları önce korunur, kararlılık sonrası ikinci migrasyonla düşürülür.

Tablo	Değişiklik	Not
songs	+ youtube_video_id text UNIQUE, + embeddable boolean DEFAULT true, + source text DEFAULT 'youtube'	spotify_track_id NULLABLE kalır (geçiş dönemi); benzersiz kimlik youtube_video_id olur
songs	spotify_track_id üzerindeki NOT NULL/unique varsayımlarının gevşetilmesi	0006'daki get_song_user_state RPC'si p_video_id parametresiyle yeni sürüme çekilir
venues	– spotify_access_token, – spotify_refresh_token, – spotify_token_expires_at, – spotify_account_id, – spotify_account_name	2. faz migrasyonda düşürülür; + player_mode text DEFAULT 'youtube' (feature flag)
now_playing	+ last_heartbeat_at timestamp, + video_id text	Oynatıcı sağlık takibi; progress_ms artık player'dan yazılır
queue	Değişiklik yok	status/priority/position/tokens_spent akışı aynen korunur
RPC'ler (0006)	get_song_user_state ve sayfa durumu RPC'leri kimlik parametresini video_id'ye taşır	Müşteri UI'nın tek dokunuş noktası

5.1 Mevcut katalog verisinin taşınması

songs tablosundaki mevcut kayıtlar (title, artist, duration_ms) korunur; her birine YouTube karşılığı Bölüm 7'deki eşleştirme işiyle doldurulur. Eşleşemeyen şarkılar embeddable=false ile işaretlenir ve müşteri aramasında gizlenir. play_count/istatistikler song_id üzerinden ilerlediği için tarihçe kaybolmaz.

6. Dosya Bazlı Değişiklik Haritası

Dosya / Modül	İşlem	Efor
lib/youtube.ts (YENİ)	searchVideos (search.list), getVideoDetails (videos.list: süre, embeddable, region), getPlaylistItems. API anahtarı sunucuda	1-1,5 gün
components/player/YouTubePlayer.tsx (YENİ)	IFrame API yükleme, loadVideoById, ENDED/onError yönetimi, ilerleme raporu, heartbeat	2 gün
/api/player/[venueId] (YENİ)	next/play/pause komutları — Spotify Connect yerine now_playing+queue günceller, player Realtime ile dinler	1 gün

Dosya / Modül	İşlem	Efor
/api/search (route değişir)	Önce songs tablosu, sonra YouTube; sonuçları songs'a upsert	0,5-1 gün
lib/spotify.ts	SİLİNİR (playNextFromQueue'nun kuyruk mantığı lib/queue.ts'e taşınır)	0,5 gün
/api/spotify/auth, /callback, /sync, /now-playing, /player, spotify-status	SİLİNİR	0,5 gün
/api/admin/spotify/playlists + import	YouTube playlist içe aktarma (playlistItems.list, playlist URL'si yapıştırarak — OAuth gerekmez)	1 gün
Admin dashboard page.tsx	spotifyConnected/now-playing polling kalkar; Player bileşeni + kuyruk yan panel düzeni	1,5-2 gün
Müşteri UI (browse/queue/song detay)	album_cover_url → YouTube thumbnail (i.ytimg.com), RPC parametre değişikliği; akış aynı	1 gün
lib/queue-fill.ts	Değişmez (yalnız embeddable=false şarkıları atlama filtresi eklenir)	0,25 gün
.env / next.config.ts	SPOTIFY_* çıkar; YOUTUBE_API_KEY girer; i.ytimg.com images domain'ine eklenir	0,25 gün

Toplam geliştirme eforu: **~10-12 iş günü**. Silinen kod (~700+ satır Spotify entegrasyonu) eklenen koddan fazladır; net karmaşıklık azalır.

7. Katalog Eşleştirme: Spotify → YouTube

En hassas iş, mevcut songs kayıtlarına doğru YouTube videosunu bulmaktır. YouTube'da aynı şarkının onlarca sürümü vardır (resmi klip, official audio, canlı, cover, sped-up). Önerilen otomatik eşleştirme hattı:

Adım	Yöntem
1. Aday arama	search.list: "{artist} {title}" + videoCategoryId=10 + videoEmbeddable=true + regionCode=TR, ilk 5 sonuç
2. Kanal önceliği	"{Artist} - Topic" kanalları (YouTube'un otomatik resmi ses kanalları) ve doğrulanmış sanatçı kanalları öne alınır — bunlar 'official audio' kalitesindedir
3. Süre doğrulaması	videos.list ile süre alınır; Spotify duration_ms $\pm$ 3 sn toleransla karşılaştırılır — canlı/uzatılmış sürümler böyle elenir
4. Başlık filtreleri	"live", "cover", "remix", "sped up", "8D" içerenler (orijinal başlıkta yoksa) cezalandırılır
5. Skorlama ve eşik	Kanal + süre + başlık benzerliği skoru; eşik altındakiler 'manuel onay' kuyruğuna düşer (super admin ekranına basit bir onay listesi eklenir)

Kota etkisi: şarkı başına ~101 birim (1 arama + 1 videos.list). Günlük 10.000 birimle günde ~95 şarkı eşleştirilir; birkaç bin şarkılık katalog için işlem geceye yayılır veya kota artışı sonrası tek seferde koşulur. Bu bir defalık maliyettir. Yeni istenen şarkılar ise kullanıcı seçimiyle eşleştigi için (müşteri aradığı sonucu kendisi seçer) sonradan eşleştirme sorunu yaşanmaz.

8. API Kota Yönetimi ve Maliyet

Uç nokta	Birim maliyeti	Kullanım yeri
search.list	100 birim	Yalnız yerel katalogda bulunamayan aramalar
videos.list	1 birim	Süre/embed doğrulaması, metadata tazeleme
playlistItems.list	1 birim	Playlist içe aktarma
IFrame Player	0 (kota tüketmez)	Tüm oynatma

Varsayılan kota 10.000 birim/gün'dür; yani önlem alınmazsa **tüm mekanlar toplamda günde ~100 YouTube aramasıyla** sınırlanır. Savunma hattı üç katmanlıdır: (1) **Yerel önce:** songs tablosunda Postgres full-text/trigram araması — olgun bir mekanda isteklerin büyük çoğunluğu kataloğa zaten girmiş şarkılardır ve 0 birime mal olur. (2) **Arama önbelleği:** aynı sorgu metni 30 gün önbelleklenir (search_cache tablosu). (3) **Kota artışı:** Google'ın YouTube API Services 'Quota extension / compliance audit' formuyla artış istenir; uygulamanın ToS uyumlu tasarımı (görünür player, resmi API'ler) bu denetimin ön şartıdır. Parasal maliyet: Data API ücretsizdir, kota parayla satın alınamaz; IFrame Player ücretsizdir. Mekan tarafında Spotify Premium aboneliği (aylık mekan başına ~200 TL) tamamen ortadan kalkar.

9. Riskler ve Azaltma Stratejileri

Risk	Olasılık / Etki	Azaltma
Reklamlar: gömülü videolarda atlama gerektiren reklam çıkabilir; mekan akışını bozar	Yüksek / Orta	Mekanın kendi YouTube Premium hesabıyla player tarayıcısında oturum açması reklamları kaldırır (hesap mekanın kendisindir, PMJ erişmez). Ürün dokümantasyonuna 'önerilen kurulum' olarak yazılır.
Embed kapalı / bölge engelli / kaldırılmış videolar	Orta / Düşük	Aramada videoEmbeddable=true filtresi + onError'da otomatik atlama + songs.embeddable işaretleme + kuyruk doldurmada filtre.
Kota tükenmesi (arama çalışmaz)	Orta / Orta	Bölüm 8'deki üç katman; kota dolarsa arama 'katalogdan seç' moduna zarifçe düşer (graceful degradation).
Admin cihazında sekme kapanması / uyku modu	Orta / Yüksek	Heartbeat + panelde çevrimdışı uyarısı; kurulum rehberinde ekran uyku ayarı kapatılır; kiosk modu (tam ekran Chrome) önerilir.
Yanlış sürüm çalması (cover/canlı)	Orta / Düşük	Topic-kanal önceliği + süre doğrulaması (Bölüm 7); müşteri zaten seçtiği videoyu görür.
ToS denetimi (Google API compliance)	Düşük / Yüksek	Görünür player, overlay yok, indirme yok, RMF'ye tam uyum; denetim öncesi self-audit yapılır.
İçerik kalitesi/uygunluğu (patlayan içerik, uygunsuz klip)	Düşük / Orta	Mevcut istek onay mekanizması korunur; admin panelde 'yalnız onaylı istekler çalar' modu zaten mevcut.

10. Hukuki Durum (ToS + Türkiye Telif)

10.1 Platform tarafı

YouTube'un standart hizmet şartları da (tıpkı Spotify gibi) bireysel/ticari olmayan kullanım içindir; bir işletmede müzik çalmak 'umuma iletim' sayılır ve platform aboneliği bu hakkı vermez. **Bu, Spotify'dan YouTube'a geçmekle ne kötüleşen ne de düzelen bir durumdur — mevcut Spotify kurulumu da aynı hukuki konumdadır.** PMJ'nin API kullanımı (Data API + IFrame Player) ise resmi ve geliştirici sözleşmesi kapsamındadır; ürünün Google nezdindeki uyumluluğu Bölüm 3.3'teki kurallara bağlıdır.

10.2 Türkiye: umuma açık mekanda müzik

5846 sayılı FSEK uyarınca umuma açık mahallerde (kafe, bar, restoran) müzik yayını için işletmenin ilgili meslek birlikleriyle (MESAM, MSG, MÜ-YAP, MÜYORBİR) lisans sözleşmesi imzalaması ve tarife bedelini ödemesi zorunludur. 2025 MESAM+MSG ortak tarifesinde 0-50 m² lokanta için yıllık bedel ~18.800 TL, 51-100 m² için ~30.800 TL seviyesindedir; bar/eğlence mekanı tarifeleri daha yüksektir. Bu yükümlülük **kaynak platformdan bağımsızdır**: radyo, Spotify veya YouTube — mekanda müzik duyuluyorsa lisans gerekir.

10.3 PMJ için önerilen hukuki konumlanma

PMJ'nin sözleşme ve kullanım şartlarına şu ilkeler yazılmalıdır: (1) PMJ bir kuyruk yönetim/etkileşim yazılımıdır, müzik yayın lisansı sağlamaz; (2) umuma iletim lisansı (meslek birlikleri) almak münhasıran mekan işletmecisinin yükümlülüğüdür; (3) mekan, YouTube hesabı ve aboneliğinin şartlarına uymayı taahhüt eder. Onboarding'e 'telif lisansınız var mı?' onay kutusu eklenmesi hem ciddiyet hem kanıt değeri taşır. **Bu bölüm hukuki danışmanlık değildir; ticarileşme öncesi fikri mülkiyet alanında çalışan bir avukattan görüş alınması önemle önerilir.**

11. Aşamalı Geçiş Planı ve Zaman Çizelgesi

Faz	İş paketleri	Süre	Çıkış kriteri
Faz 0 — Hazırlık	Google Cloud projesi + YouTube Data API anahtarı; .env düzeni; 0008 migrasyonu (kolon ekleme, flag); i.ytimg.com image domain	1-2 gün	Staging'de arama ucu YouTube'dan sonuç dönüyor
Faz 1 — Çekirdek	lib/youtube.ts; YouTubePlayer bileşeni; /api/player; arama route'unun yerel-önce mantığı; now_playing Realtime yayını	1 hafta	Test mekanında uçtan uca: iste → onayla → çal → bitince sıradaki
Faz 2 — Panel & müşteri UI	Admin dashboard yeniden düzeni (player + kuyruk); müşteri 'şimdi çalışıyor' ekranının Realtime'a geçişi; playlist içe aktarma; RPC güncellemeleri	1 hafta	Tüm ekranlar Spotify uçlarına istek atmıyor
Faz 3 — Katalog & pilot	Eşleştirme betiği (Bölüm 7); manuel onay ekranı; 1 gerçek mekanda pilot (player_mode='youtube'); heartbeat/uyarılar	1 hafta	Pilot mekan 7 gün kesintisiz; şikayet eşigi altında
Faz 4 — Yaygınlaştırma & temizlik	Tüm mekanlar flag ile geçirilir; Spotify route'ları ve kolonları silinir (0009); kota artış başvurusu; dokümantasyon	3-4 gün	Kod tabanında SPOTIFY_* kalmadı; kota başvurusu iletildi

Geri dönüş planı: Faz 3 sonuna kadar Spotify kodu ve kolonları silinmediği için venues.player_mode='spotify' yapılarak mekan bazında anında geri dönülebilir. Kalıcı silme yalnızca pilot başarı kriterleri sağlandıktan sonra yapılır.

12. Sonuç ve Öneriler

YouTube Music'e geçiş teknik olarak mümkün değildir; YouTube (video) altyapısına geçiş ise hem mümkün hem de mevcut mimariden daha sağlamdır. Kuyruk zaten Supabase'de olduğundan geçiş, 'çalma motorunun' değiştirilmesinden ibarettir: OAuth/token/cihaz/polling katmanı silinir, yerine panel içi görünür bir player ve olay tabanlı ilerleme gelir. Mekan kurulum sürtünmesi (Premium hesap, hesap bağlama, 25 kullanıcı limiti) ortadan kalkar — ölçeklenme önündeki asıl engel budur ve çözülür.

Kritik başarı faktörleri: **(1)** video görünürlüğü kuralını tasarımın merkezine koymak (klip ekranını ürün özelliğine çevirmek), **(2)** arama kotasını yerel-önce mimariyle korumak ve erkenden kota artışına başvurmak, **(3)** reklamsız deneyim için mekanlara kendi YouTube Premium oturumlarını önermek, **(4)** telif yükümlülüğünü sözleşmeyle net biçimde mekana bırakmak ve avukat görüşü almak. Bu dört koşul sağlandığında geçiş, 3-4 haftalık kontrollü bir projeye tamamlanabilir.

Kaynaklar: YouTube API Services — Required Minimum Functionality & Developer Policies (developers.google.com/youtube/terms), YouTube IFrame Player API Reference, YouTube Data API v3 kota dokümantasyonu, ytmusicapi dokümantasyonu (readthedocs), MESAM-MSG 2025 Genel Lisanslama Ortak Tarifesi (mesam.org.tr), MESAM SSS — işyerinde müzik yayını. Kod tabanı analizi: PMJ deposu (lib/spotify.ts, app/api/spotify/, supabase/migrations/*), 10 Temmuz 2026.*